Homework 5

ASEN 5090 Introduction to GNSS

Justin Le, Elsa Carreras

October 16, 2026

Problem 1:

(a.)

To make NIST the a priori position we set it as the initial guess in the code in Appendix 2,

$$\phi = 39.9951^\circ, \quad \lambda = -105.2626^\circ$$

$$R_L = \begin{bmatrix} -\sin\lambda & \cos\lambda & 0 \\ -\sin\phi\cos\lambda & -\sin\phi\sin\lambda & \cos\phi \\ \cos\phi\cos\lambda & \cos\phi\sin\lambda & \sin\phi \end{bmatrix}$$

$$R_L = \begin{bmatrix} 0.9647 & -0.2632 & 0 \\ 0.1692 & 0.6201 & 0.7661 \\ -0.2017 & -0.7391 & 0.6427 \end{bmatrix}$$

(b.)

PRN	PIF (m)	R1 (m)	EL (deg)	AZ (deg)	BSV (m)	RELAT (m)	TROP (m)
3	22503863.87	22598412.10	33.0	-165.0	94613.41	-4.47	3.66
4	20218540.40	20237719.09	80.2	-0.8	19238.10	2.59	2.03
7	23384557.16	23319494.85	29.1	-91.0	-64990.19	-8.94	4.10
8	24732437.24	24843347.95	7.2	159.2	110977.66	7.23	15.14
9	21285686.19	21494983.06	46.5	-47.3	209359.06	1.20	2.76
16	20246092.21	20341487.93	62.6	79.1	95458.55	0.20	2.25
26	22303327.89	22186498.61	34.9	50.9	-116761.08	-5.40	3.49
27	23541485.58	23541578.19	19.1	126.9	152.17	7.99	6.05
31	24054577.04	24016379.48	16.2	68.3	-38123.84	-7.42	7.08

Table 1: GPS satellite observations and computed corrections

Problem 2:

(a.)

PRN	Prefit residual
3	57.0469 m
4	59.9685 m
7	59.0737 m
8	59.0396 m
9	60.6348 m
16	60.7752 m
26	59.3047 m
27	61.4985 m
31	59.2230 m

Table 2: Prefit residual for all PRNs

(b.)

$$G = \begin{bmatrix} 0.4566 & 0.8477 & 0.2700 & 1 \\ 0.1722 & 0.6222 & -0.7637 & 1 \\ 0.9437 & 0.1386 & -0.3004 & 1 \\ -0.1585 & 0.7599 & 0.6305 & 1 \\ 0.5555 & 0.1127 & -0.8238 & 1 \\ -0.2722 & 0.7211 & -0.6371 & 1 \\ -0.5862 & 0.2693 & -0.7641 & 1 \\ -0.5672 & 0.7926 & 0.2236 & 1 \\ -0.8644 & 0.2209 & -0.4516 & 1 \end{bmatrix} \begin{matrix} \text{PRN 3} \\ \text{PRN 4} \\ \text{PRN 7} \\ \text{PRN 8} \\ \text{PRN 9} \\ \text{PRN 16} \\ \text{PRN 26} \\ \text{PRN 27} \\ \text{PRN 31} \end{matrix}$$

(c.)

State vector corrections:

$$\begin{bmatrix} \delta x \\ \delta y \\ \delta z \\ \delta b \end{bmatrix} = \begin{bmatrix} -0.5806 \text{ m} \\ 0.6872 \text{ m} \\ -1.0809 \text{ m} \\ 58.9409 \text{ m} \end{bmatrix}$$

(d.)

PRN	Post-fit Residual (m)
3	-1.9197
4	-0.1255
7	0.2607
8	0.1659
9	1.0484
16	0.4921
26	-0.9876
27	1.9253
31	-0.8597

Table 3: Post-fit pseudorange residuals

We observe from comparing post fit residuals to prefit residuals that our post fit are much smaller. Our biggest residuals from the post fit are PRN 3 and PRN 27. These maybe indicating noise, multipath, or modeling error. The ones that are very close to zero such as PRN 7 or 8 fit the linear model we used well.

(e.)

For this problem we removed 3, 8, 27, and 31. For our new state vector corrections we get:

$$\begin{bmatrix} \delta x \\ \delta y \\ \delta z \\ \delta b \end{bmatrix} = \begin{bmatrix} 0.6495 \text{ m} \\ 1.5605 \text{ m} \\ -2.5321 \text{ m} \\ 57.5983 \text{ m} \end{bmatrix}$$

Thus our post fit residuals update as well:

PRN	Post-fit Residual (m)
4	-0.6464
7	-0.1145
9	0.4137
16	0.6151
26	-0.2679

Table 4: Post-fit pseudorange residuals after removing PRNs 3, 8, 27, and 31

So this shows when we remove the low elevation satellites and remove outlier post fit residuals our answer will fit the linearized model much more. This does not mean a more accurate position since we are just limiting our data. To check this we did a position error check between the two different state vector corrections and found Before removing satellites: position error = 1.4063 m and after removing satellites: Position error = 3.0444 m.

Matlab position error check code:

```
new_position_estimate=siteguess+state_vector_corrections(1:3,:)';
straightline2=norm(NIST_ECEF-new_position_estimate)
```

Problem 3:

(a.)

Using the a priori guess of [40°, −105°, 1600 m] in LLA coordinates, we can convert to ECEF coordinates of $\vec{r}_0 = [-1.2666 \times 10^6,\ -4.7272 \times 10^6,\ 4.0790 \times 10^6]$ m. We find an error vector of $\Delta\vec{r}_0 = \vec{r}_{\text{NIST}} - \vec{r}_0 = \boxed{[-2.1755 \times 10^4,\ 5.480 \times 10^3,\ -3.89 \times 10^2] \text{ m}}$. Finding the straight line distance between the guess and the true location of NIST by taking the norm, we find that $\boxed{r_{\text{error}} = 2.2438 \times 10^4 \text{ m}}$.

(b.)

We repeat what was done in Part 2 to find the state vector corrections.

PRN	PIF (m)	R1 (m)	EL (deg)	AZ (deg)	BSV (m)	RELAT (m)	TROP (m)
3	22503863.87	22603816.43	33.0	-164.6	94613.41	-4.47	3.67
4	20218540.40	20237772.61	80.2	-2.2	19238.10	2.59	2.03
7	23384557.16	23339151.75	28.8	-90.8	-64990.19	-8.94	4.13
8	24732437.24	24835990.46	7.2	159.4	110977.66	7.23	15.01
9	21285686.19	21506139.37	46.3	-47.3	209359.06	1.20	2.76
16	20246092.21	20331377.45	62.8	79.2	95458.55	0.20	2.25
26	22303327.89	22171978.43	35.1	50.9	-116761.08	-5.40	3.47
27	23541485.58	23524986.02	19.3	127.1	152.17	7.99	6.00
31	24054577.04	23996189.52	16.4	68.4	-38123.84	-7.42	7.00

Table 5: GPS satellite observations and computed corrections at a priori position [40°, −105°, 1600 m]

PRN	Prefit Residual (m)
3	-5347.2893
4	6.4514
7	-19597.8504
8	7416.6538
9	-11095.6848
16	10171.2658
26	14579.5039
27	16653.7118
31	20249.2694

Table 6: Prefit residuals at the a priori position [40°, −105°, 1600 m]

Notice how due to our much more incorrect a priori position, the prefit residuals are not in the 10s of km.

$$G = \begin{bmatrix} 0.4575 & 0.8473 & 0.2699 & 1 \\ 0.1733 & 0.6219 & -0.7636 & 1 \\ 0.9438 & 0.1382 & -0.3001 & 1 \\ -0.1576 & 0.7599 & 0.6307 & 1 \\ 0.5562 & 0.1124 & -0.8234 & 1 \\ -0.2712 & 0.7212 & -0.6374 & 1 \\ -0.5856 & 0.2693 & -0.7645 & 1 \\ -0.5667 & 0.7929 & 0.2238 & 1 \\ -0.8643 & 0.2208 & -0.4520 & 1 \end{bmatrix} \begin{matrix} \text{PRN 3} \\ \text{PRN 4} \\ \text{PRN 7} \\ \text{PRN 8} \\ \text{PRN 9} \\ \text{PRN 16} \\ \text{PRN 26} \\ \text{PRN 27} \\ \text{PRN 31} \end{matrix}$$

Notice how the G matrix had minimal changes, likely because of how far the GPS satellites are having small impacts on the geometry relative to the distance of this new error.

$$\delta\hat{\vec{x}} = \begin{bmatrix} -21754 \text{ m} \\ 5493 \text{ m} \\ -394 \text{ m} \\ 59 \text{ m} \end{bmatrix}$$

PRN	**Post-fit Residual (m)**
3	-1.7783
4	0.7183
7	-2.6468
8	3.1942
9	3.3499
16	-0.8310
26	0.1678
27	-0.6009
31	-1.5732

Table 7: Post-fit residuals after the first iteration from the a priori position [40°, −105°, 1600 m]

(c.)

We then update the position estimate based on these corrections,

$$\vec{r}_1 = \begin{bmatrix} -1.2884 \times 10^6 \text{ m} \\ -4.7217 \times 10^6 \text{ m} \\ 4.0786 \times 10^6 \text{ m} \end{bmatrix}$$

$$\Delta\vec{r}_1 = \vec{r}_{\text{NIST}} - \vec{r}_1 = \begin{bmatrix} -1.8102 \text{ m} \\ -13.0606 \text{ m} \\ 5.6371 \text{ m} \end{bmatrix}$$

$$\boxed{r_{\text{error}} = 14.3 \text{ m}}$$

notice how this error is substantially smaller than what was calculated in Part (a.).

(f.)

We now iterate this process until the solution converges to under 1 cm. The final resulting state vector correction would be,

Iteration	δx (m)	δy (m)	δz (m)	δb (m)	$\lvert\delta\hat{\vec{x}}\rvert$ (m)	x (m)	y (m)	z (m)	b (m)
1	−21753.62	5492.67	−394.32	58.95	22439.88	−1288396.76	−4721683.87	4078619.71	58.95
2	−2.39	−12.37	4.56	−0.01	13.40	−1288399.15	−4721696.24	4078624.27	58.94
3	0.00	0.00	0.00	0.00	0.00	−1288399.15	−4721696.24	4078624.27	58.94

Table 8: State vector corrections and updated position

(g.)

From Table 8, we find the clock bias estimate to be about $\boxed{58.94 \text{ m}}$. We then find that the final $\Delta\vec{r}_f = \vec{r}_{\text{NIST}} - \vec{r}$ is $\boxed{\Delta\vec{r}_{f,\text{ECEF}} = [0.58,\ -0.69,\ 1.08] \text{ m}}$.

We also calculate the post fit residuals, here they would be,

$$\delta\vec{\rho}_{\text{post}} = \begin{bmatrix} -1.9197 \\ -0.1255 \\ 0.2607 \\ 0.1660 \\ 1.0484 \\ 0.4921 \\ -0.9876 \\ 1.9253 \\ -0.8597 \end{bmatrix} \text{ m}$$

Examining the postfit residuals, we find that they fit the model to≈2 meters, which makes it pretty good, given the clock and atmospheric corrections we implemented previously. The largest residual of these correspond to PRN 3 and PRN 27, likely a result of some clock error due to those satellites specifically since they have relatively high elevations. The sum of all these residuals also add up to 0.

(h.)

We can convert to ENU frame with,

$$R_L = \begin{bmatrix} -\sin\lambda & \cos\lambda & 0 \\ -\sin\phi\cos\lambda & -\sin\phi\sin\lambda & \cos\phi \\ \cos\phi\cos\lambda & \cos\phi\sin\lambda & \sin\phi \end{bmatrix}$$

$$\Delta\vec{r}_{f,\text{ENU}} = R_L\,\Delta\vec{r}_{f,\text{ECEF}}$$

$$\therefore \boxed{\Delta\vec{r}_{f,\text{ENU}} = [0.74, 0.50, 1.09] \text{ m}}$$

(i.)

We want to find HDOP and VDOP. This can be done by taking the G matrix and converting it to the ENU frame, then using the resulting G matrix to calculate for an $\tilde{H}$ matrix. Where HDOP and VDOP can be found as,

$$\tilde{R} = \begin{bmatrix} R_L & 0 \\ 0 & 1 \end{bmatrix}$$

$$\tilde{G} = G\,\tilde{R}^T$$

$$\tilde{H} = \left(\tilde{G}^T\tilde{G}\right)^{-1}$$

$$HDOP = \sqrt{\tilde{H}(1,1) + \tilde{H}(2,2)}$$

$$VDOP = \sqrt{\tilde{H}(3,3)}$$

$$\therefore \boxed{HDOP = 0.9804}$$

$$\therefore \boxed{VDOP = 1.5417}$$

Problem 4:

Repeating what was done in Problem 3, we initialize the a priori guess to, $[16.776, -3.009, 261]$ in Timbuktu, Mali. Converting to ECEF would be $\vec{r}_{\text{far,ECEF}} = [6.1002\times10^6,\ -3.207\times10^5,\ 1.8292\times10^6]$ m.

Running the iteration loop like Problem 3 we obtain,

Iteration	δx (m)	δy (m)	δz (m)	δb (m)	$\lvert\delta\hat{\vec{x}}\rvert$ (m)	x (m)	y (m)	z (m)	b (m)
1	−8508450.91	−4190990.18	2283892.69	1143427.74	9822511.30	−2408232.87	−4511649.92	4113080.87	1143427.74
2	1124531.87	−176386.15	−47301.48	−1144566.03	1614915.73	−1283701.00	−4688036.07	4065779.39	−1138.29
3	−4701.52	−33689.54	12855.47	1233.40	36385.08	−1288402.52	−4721725.61	4078634.85	95.11
4	3.37	29.36	−10.58	−36.17	47.90	−1288399.15	−4721696.24	4078624.27	58.94
5	0.00	0.00	0.00	0.00	0.00	−1288399.15	−4721696.24	4078624.27	58.94

Table 9: State vector correction and updated position starting at Timbuktu

Examining the final result after 5 iterations for the solution to converge, we find that we do get the same final result as we did in Part 3. Ending up at the same positions with what we found earlier at NISTS's location. The reason why we get the same results despite initializing so far away is because of the distance to GPS satellites and their line of sights. Because satellites are still about 20,000 km up, the line of sight error relative to these satellites are not major enough to warrant large divergences or errors, allowing for errors to collapse relatively quickly as seen in Part 3. As a result, even though initialized in Timbuktu, it just takes 2 more iterations for the solution to converge on NIST.

MATLAB Code

```
%% ASEN 5090 GPS/GNSS
%% Justin Le
%% HW5 Main Script
clc;clear;close all

%% Reading in data
% Reading in rinex
rinexData = rinexread('NIST00USA_R_20262310000_01D_30S_MO.rnx');
rinexDataGPS = rinexData.GPS;

% Read in ephemeris
ephemeris = read_clean_GPSbroadcast('brdc2310.26n', true);

%% Part 1 Filling out table
disp('<strong>Part 1</strong>')
NIST_ECEF = [-1288398.567 -4721696.932 4078625.350];

PRN_list = [3,4,7,8,9,16,26,27,31];
for PRN = PRN_list
    PRNid = sprintf('PRN%d', PRN);
    [PIF.(PRNid), R1.(PRNid), EL.(PRNid), AZ.(PRNid), BSV.(PRNid), RELAT.(PRNid), TROP.(
        PRNid), SATPOS.(PRNid)] = singlepointsolution(PRN,rinexDataGPS, ephemeris,
        NIST_ECEF);
end

% Obtain only the very first values and put in a struct, also the first sat
% pos for G matrix
first_vals = zeros(length(PRN_list), 7);
satPos_first = zeros(length(PRN_list), 3);
for i = 1:length(PRN_list)
    PRNid = sprintf('PRN%d', PRN_list(i));
    first_vals(i,:) = [PIF.(PRNid)(1), R1.(PRNid)(1), EL.(PRNid)(1), AZ.(PRNid)(1), BSV.(
        PRNid)(1), RELAT.(PRNid)(1), TROP.(PRNid)(1)];
    satPos_first(i,:) = SATPOS.(PRNid)(1,:);

    first.PIF.(PRNid) = PIF.(PRNid)(1);
    first.R1.(PRNid) = R1.(PRNid)(1);
    first.EL.(PRNid) = EL.(PRNid)(1);
    first.AZ.(PRNid) = AZ.(PRNid)(1);
    first.BSV.(PRNid) = BSV.(PRNid)(1);
    first.RELAT.(PRNid) = RELAT.(PRNid)(1);
    first.TROP.(PRNid) = TROP.(PRNid)(1);
end
first_val_table = array2table(first_vals, 'VariableNames', {'PIF','R1','EL','AZ','BSV','
    RELAT','TROP'}, ...
    'RowNames', compose('PRN%d', PRN_list'));
disp(first_val_table)

%% Part 2 Calculate least square point solution
disp('<strong>Part 2</strong>')
[G, dx_hat, delta_rho_post] = compute_least_point_solution(PRN_list, first, satPos_first,
     NIST_ECEF);
```

```
% Redoing this without lower elevation sats/large postfit residuals
% Removing low elevation, PRN8
% Removing large postfit residuals, PRN3, PRN27

post_PRN_list = [4,7,9,16,26,31];
for PRN = post_PRN_list
    PRNid = sprintf('PRN%d', PRN);
    [PIF.(PRNid), R1.(PRNid), EL.(PRNid), AZ.(PRNid), BSV.(PRNid), RELAT.(PRNid), TROP.(
        PRNid), SATPOS.(PRNid)] = singlepointsolution(PRN,rinexDataGPS, ephemeris,
        NIST_ECEF);
end

disp('<strong>Part 2 adjusting PRNs</strong>')
% Obtain only the very first values and put in a struct, also the first sat
% pos for G matrix
first_vals = zeros(length(post_PRN_list), 7);
satPos_first = zeros(length(post_PRN_list), 3);
for i = 1:length(post_PRN_list)
    PRNid = sprintf('PRN%d', post_PRN_list(i));
    first_vals(i,:) = [PIF.(PRNid)(1), R1.(PRNid)(1), EL.(PRNid)(1), AZ.(PRNid)(1), BSV.(
        PRNid)(1), RELAT.(PRNid)(1), TROP.(PRNid)(1)];
    satPos_first(i,:) = SATPOS.(PRNid)(1,:);

    first.PIF.(PRNid) = PIF.(PRNid)(1);
    first.R1.(PRNid) = R1.(PRNid)(1);
    first.EL.(PRNid) = EL.(PRNid)(1);
    first.AZ.(PRNid) = AZ.(PRNid)(1);
    first.BSV.(PRNid) = BSV.(PRNid)(1);
    first.RELAT.(PRNid) = RELAT.(PRNid)(1);
    first.TROP.(PRNid) = TROP.(PRNid)(1);
end
first_val_table = array2table(first_vals, 'VariableNames', {'PIF','R1','EL','AZ','BSV','
    RELAT','TROP'}, ...
    'RowNames', compose('PRN%d', post_PRN_list'));
disp(first_val_table)

[G, dx_hat, delta_rho_post] = compute_least_point_solution(post_PRN_list, first,
    satPos_first, NIST_ECEF);

% Find and print error

%% Part 3
disp('<strong>Part 3</strong>')
% given an initial guess
r0_ECEF = lla2ecef([40,-105,1600],'WGS84');

% Error vector
delta_r0 = NIST_ECEF - r0_ECEF;

% Finding straightline
disp('<strong>Striaght line error</strong>')
disp(norm(delta_r0))
```

```
% b. Find state vector corrections
PRN_list = [3,4,7,8,9,16,26,27,31];
for PRN = PRN_list
    PRNid = sprintf('PRN%d', PRN);
    [PIF.(PRNid), R1.(PRNid), EL.(PRNid), AZ.(PRNid), BSV.(PRNid), RELAT.(PRNid), TROP.(
        PRNid), SATPOS.(PRNid)] = singlepointsolution(PRN,rinexDataGPS, ephemeris, r0_ECEF
        );
end

% Obtain only the very first values and put in a struct, also the first sat
% pos for G matrix
first_vals = zeros(length(PRN_list), 7);
satPos_first = zeros(length(PRN_list), 3);
for i = 1:length(PRN_list)
    PRNid = sprintf('PRN%d', PRN_list(i));
    first_vals(i,:) = [PIF.(PRNid)(1), R1.(PRNid)(1), EL.(PRNid)(1), AZ.(PRNid)(1), BSV.(
        PRNid)(1), RELAT.(PRNid)(1), TROP.(PRNid)(1)];
    satPos_first(i,:) = SATPOS.(PRNid)(1,:);

    first.PIF.(PRNid) = PIF.(PRNid)(1);
    first.R1.(PRNid) = R1.(PRNid)(1);
    first.EL.(PRNid) = EL.(PRNid)(1);
    first.AZ.(PRNid) = AZ.(PRNid)(1);
    first.BSV.(PRNid) = BSV.(PRNid)(1);
    first.RELAT.(PRNid) = RELAT.(PRNid)(1);
    first.TROP.(PRNid) = TROP.(PRNid)(1);
end
first_val_table = array2table(first_vals, 'VariableNames', {'PIF','R1','EL','AZ','BSV','
    RELAT','TROP'}, ...
    'RowNames', compose('PRN%d', PRN_list'));

[G, dx_hat, delta_rho_post] = compute_least_point_solution(PRN_list, first, satPos_first,
     r0_ECEF);

% c. Update estimate from state estimate corrections
r1_ECEF = r0_ECEF + dx_hat(1:3)';
delta_r1 = NIST_ECEF - r1_ECEF;
disp('<strong>Striaght line error</strong>')
disp(norm(delta_r0))
disp(norm(delta_r1))

% d. Iterate
% Initial delta, start from the a priori guess with no clock bias estimate
[r_est, b_est, G, delta_rho_post, delta_r_f_ECEF, delta_r_f_ENU, HDOP, VDOP,
    iteration_count] = ...
    least_squares_correction(PRN_list, r0_ECEF, NIST_ECEF, rinexDataGPS, ephemeris);

for k = 1:size(iteration_count,1)
    fprintf('Iter %d: |dx| = %.2f m, r = [%.2f, %.2f, %.2f] m, bsv = %.2f, dx = [%.2f,
        %.2f, %.2f, %.2f]\n', iteration_count(k,:));
end

% Find the delta r error from NIST
disp('<strong>delta_r_f ECEF</strong>')
```

```
disp(delta_r_f_ECEF)

% Display the postfit residuals as well
disp('<strong>Postfit Residuals</strong>')
disp(delta_rho_post) %they do not seem to change much?

% Report delta_r_f_ENU
disp('<strong>delta_r_f ENU</strong>')
disp(delta_r_f_ENU)

% Compute HDOP and VDOP
disp('<strong>HDOP and VDOP</strong>')
fprintf('HDOP: %.4f\n', HDOP);
fprintf('VDOP: %.4f\n', VDOP);


%% Part 4:
r_far_ECEF = lla2ecef([16.776, -3.009, 261]); % I am going to Timbuktu
[r_est, b_est, G, delta_rho_post, delta_r_f_ECEF, delta_r_f_ENU, HDOP, VDOP,
    iteration_count] = ...
    least_squares_correction(PRN_list, r_far_ECEF, NIST_ECEF, rinexDataGPS, ephemeris);

for k = 1:size(iteration_count,1)
    fprintf('Iter %d: |dx| = %.2f m, r = [%.2f, %.2f, %.2f] m, bsv = %.2f, dx = [%.2f,
        %.2f, %.2f, %.2f]\n', iteration_count(k,:));
end

% Find the delta r error from NIST
disp('<strong>delta_r_f ECEF</strong>')
disp(delta_r_f_ECEF)

% Display the postfit residuals as well
disp('<strong>Postfit Residuals</strong>')
disp(delta_rho_post) %they do not seem to change much?

% Report delta_r_f_ENU
disp('<strong>delta_r_f ENU</strong>')
disp(delta_r_f_ENU)

% Compute HDOP and VDOP
disp('<strong>HDOP and VDOP</strong>')
fprintf('HDOP: %.4f\n', HDOP);
fprintf('VDOP: %.4f\n', VDOP);
```

```
function [PIF, R1, EL, AZ, BSV, RELAT, TROP, SATPOS] = singlepointsolution_spare(PRN,
    rinexDataGPS, ephemeris, user_ECEF)
% Using clock corrections, relativistic corrections, tropospheric
% corrections, ionospheriec corrections, output a pseudorange

% With PRN number a, extract rinex and ephemeris
PRN_number = PRN;
PRN_index = rinexDataGPS.SatelliteID == PRN_number;
PRN_GPS_rinexData = rinexDataGPS(PRN_index,:);

rho1 = PRN_GPS_rinexData.C1C;
rho2 = PRN_GPS_rinexData.C2W;
% f1 corresponds to rho1
% f2 corresponds to rho2
f1 = 1575.42e6; % C1 frequency
f2 = 1227.6e6; % C2 frequency
% f5 = 1176.45e6; % C5 frequency
% Finding ionosphere free pseudorange
[PIF, ~] = ionocorr (rho1, f1, rho2, f2);

% Finding expected range, azimuth, elevation
[GPS_week, GPS_TOW] = utc2gpstime(PRN_GPS_rinexData.Time);
[R1, AZ, EL, SATPOS] = compute_expected_range(ephemeris, GPS_week, GPS_TOW, PRN_number,
    user_ECEF);

% Finding relative clock corrections
% Compute satellite clock bias and relativistic correction
[~,~,~,satClkCorr,~,~,relCorr] = eph2pvt2025(ephemeris, [GPS_week GPS_TOW], PRN_number);
BSV = satClkCorr;
RELAT = relCorr;

% Apply tropospheric correction using elevation and receiver position
% Assuming zd = 2 fixed for this assignment
zd = 2;
TROP = tropomodel(zd, EL);
end
```

```
function [G, dx_hat, delta_rho_post] = compute_least_point_solution(PRN_list, epoch_vals,
     satPos, user_ECEF)

%% Part 2 Calculate least square point solution
% Computing prefit residuals
for PRN = PRN_list
    PRNid = sprintf('PRN%d', PRN);
    delta_rho.(PRNid) = epoch_vals.PIF.(PRNid) - (epoch_vals.R1.(PRNid) - epoch_vals.BSV
        .(PRNid) - epoch_vals.RELAT.(PRNid) + epoch_vals.TROP.(PRNid));
end
disp(delta_rho)

% Constructing G matrix
% Initial receiver position
x0 = user_ECEF(1);
y0 = user_ECEF(2);
z0 = user_ECEF(3);

% Satellite positions
xs = satPos(:,1);
ys = satPos(:,2);
zs = satPos(:,3);

% Finding R0
R0 = sqrt((xs - x0).^2 + (ys - y0).^2 + (zs - z0).^2);

% Geometry matrix
G = [-(xs - x0)./R0, -(ys - y0)./R0, -(zs - z0)./R0, ones(length(PRN_list),1)];
disp('G Matrix')
disp(G)

% Computing least squares solution
% Get structure into a vector
delta_rho = cell2mat(struct2cell(delta_rho));
dx_hat = (G'*G)\(G'*delta_rho);

delta_x = dx_hat(1);
delta_y = dx_hat(2);
delta_z = dx_hat(3);
delta_b = dx_hat(4);

disp('Least Squares Solution')
disp(dx_hat)

% Compute post fit residuals
delta_rho_post = delta_rho - G*dx_hat;
disp('Post Fit Residuals')
disp(delta_rho_post)
end
```

```
function [r_est, b_est, G, delta_rho_post, delta_r_f_ECEF, delta_r_f_ENU, HDOP, VDOP,
    iteration_count, dx_hat] = least_squares_correction(PRN_list, r_apriori_ECEF,
    NIST_ECEF, rinexDataGPS, ephemeris)
% Iterates the least squares point solution from an a priori position

% Inits
r_est = r_apriori_ECEF;
b_est = 0;
delta_threshold = Inf;
iter = 0;
iteration_count = zeros(0,10);
while delta_threshold > 1E-2 % Solution converges under 1 cm
    iter = iter + 1;

    for PRN = PRN_list
        PRNid = sprintf('PRN%d', PRN);
        [PIF.(PRNid), R1.(PRNid), EL.(PRNid), AZ.(PRNid), BSV.(PRNid), RELAT.(PRNid), TROP
            .(PRNid), SATPOS.(PRNid)] = singlepointsolution(PRN,rinexDataGPS, ephemeris,
            r_est);
    end
    for i = 1:length(PRN_list)
        PRNid = sprintf('PRN%d', PRN_list(i));
        satPos_first(i,:) = SATPOS.(PRNid)(1,:);
        first.PIF.(PRNid) = PIF.(PRNid)(1) - b_est; % Remove clock bias
        first.R1.(PRNid) = R1.(PRNid)(1);
        first.BSV.(PRNid) = BSV.(PRNid)(1);
        first.RELAT.(PRNid) = RELAT.(PRNid)(1);
        first.TROP.(PRNid) = TROP.(PRNid)(1);
    end

    [G, dx_hat, delta_rho_post] = compute_least_point_solution(PRN_list, first,
        satPos_first, r_est);

    % Update position and clock bias estimates
    r_est = r_est + dx_hat(1:3)';
    b_est = b_est + dx_hat(4);
    delta_threshold = norm(dx_hat);

    iteration_count(iter,:) = [iter, delta_threshold, r_est, b_est, dx_hat'];
end

% Find the delta r error from NIST
delta_r_f_ECEF = NIST_ECEF - r_est;

% Report delta_r_f_ENU
% Convert from ECEF to ENU
% Get ECEF to LLA
NIST_LLA = ecef2lla(NIST_ECEF);
lambda = NIST_LLA(2);
phi = NIST_LLA(1);
R_L = ECEF2ENU(phi,lambda);

delta_r_f_ENU = R_L*delta_r_f_ECEF';
```

```
% Compute HDOP and VDOP
% Since find H matrix, need G in ENU
R_tilde = blkdiag(R_L, 1); % accounting for clock bias part
G_ENU = G*R_tilde';

H_ENU = inv(G_ENU'*G_ENU);
HDOP = sqrt(H_ENU(1,1)+H_ENU(2,2));
VDOP = sqrt(H_ENU(3,3));

end
```

```
function [R_expected, AZ_expected, EL_expected,satPos_ECEF_expected] =
    compute_expected_range(ephemeris, week, tow, PRN_number, receiver_ECEF)
% Computes expected range by iteration

% Constants for the alg
c = 2.99792458e8; % speed of light, m/s
wE = 7.2921151467e-5; % Earth rotation rate, rad/s

% Step 1: Satellite ECEF position at Tr initially.
Tr = tow + week*604800;
[~, satPos, ~, ~] = eph2pvt2025(ephemeris, [week tow], PRN_number);

% Step 2: Geometric range using NIST_ECEF and satPos at Tr
% Reuse azel range for convinience
[AZ, EL, R] = compute_azelrange(receiver_ECEF, satPos);

% Iterate until range converges
% Step 3: initial time of transmission
Tt = Tr - (R/c);
R_expected = R;

for iter = 1:5
    % Step 3: time of transmission in gps week number and TOW
    Tt_week = floor(Tt/604800);
    Tt_tow = Tt - Tt_week*604800;

    % Step 4: satellite ECEF position at Tt
    [~, satPos_Tt, ~, ~] = eph2pvt2025(ephemeris, [Tt_week Tt_tow], PRN_number);

    % Step 5: rotate sat position into the ECEF frame
    % Sat position from receiver
    x = satPos_Tt(:,1);
    y = satPos_Tt(:,2);
    z = satPos_Tt(:,3);

    phi = wE .* (Tr - Tt);
    x_ECEF = x.*cos(phi) + y.*sin(phi);
    y_ECEF = -x.*sin(phi) + y.*cos(phi);

    satPos_Tt_ECEF = [x_ECEF, y_ECEF, z];

    % Step 6: recompute geometric range with the rotated sat position
    [AZ_new, EL_new, R_new] = compute_azelrange(receiver_ECEF, satPos_Tt_ECEF);

    % Convergeance condition
    if max(abs(R_new - R_expected)) < 1e-4
        R_expected = R_new;
        AZ_expected = AZ_new;
        EL_expected = EL_new;
        satPos_ECEF_expected=satPos_Tt_ECEF;
        break
    end
    R_expected = R_new;
    AZ_expected = AZ_new;
```

```
    EL_expected = EL_new;
    satPos_ECEF_expected=satPos_Tt_ECEF;
    Tt = Tr - R_expected/c; % Step 3 repeat

end

end
```

```
function [AZ, EL, RANGE] = compute_azelrange(userECEF, satECEF)
% satECEF is ECEF vector of sat
% userECEF is ECEF vector of the user

% Calculate LLA from userECEF
lla = ecef2lla(userECEF,'WGS84');
ref_lat = lla(1);
ref_lon = lla(2);
altitude = lla(3);
% Take difference of ECEF vectors, conver to ENU depending on ref angle
r_ECEF = satECEF-userECEF;

% Calculting range.
RANGE = sqrt(sum(r_ECEF.^2,2));

% Converting to ENU frame
r_ENU = (ECEF2ENU(ref_lat,ref_lon)*r_ECEF')';
x_E = r_ENU(:,1);
x_N = r_ENU(:,2);
x_U = r_ENU(:,3);

% Calclating azimuth and elevation
AZ = atan2d(x_E,x_N);
EL = asind(x_U./sqrt(sum(r_ENU.^2,2)));

end
```

```
function C_ECEF2ENU = ECEF2ENU(ref_lat_deg, ref_lon_deg)

phi = ref_lat_deg;
lambda = ref_lon_deg;

R1 = [1 0 0;
    0 cosd(90-phi) sind(90-phi);
    0 -sind(90-phi) cosd(90-phi)];

R2 = [-sind(lambda) cosd(lambda) 0;
    -cosd(lambda) -sind(lambda) 0;
    0 0 1];

C_ECEF2ENU = R1*R2;

end
```

```
function [PRIF12, iono] = ionocorr (C1C, f1, C2L, f2)

rho_1 = C1C;
rho_2 = C2L;

iono = ((f2^2)/(f1^2 - f2^2)) * (rho_2-rho_1);

PRIF12 = ((f1^2)/(f1^2 - f2^2))*rho_1 - ((f2^2)/(f1^2 - f2^2))*rho_2;

end
```

```
function [tropo] = tropomodel(zd, elevation)

% TODO: check model, this is currently a mapping function I think?

tropo = zd * (1 ./ sqrt(1 - (cosd(elevation)/1.001).^2 ));

end
```

1 Appendix 2 Matlab Code for problems 1 and two

MATLAB Code

```
clc; clear; close all;
%Author: Elsa Carreras
%Problem 1

load('NIST_RINEX_GPS.mat');
load('NIST_GPS_broadcast.mat');

PRN_number = [3; 4;7;8;9;16;26;27;31];

NIST_ECEF = [-1288398.567 -4721696.932 4078625.350];
siteguess=NIST_ECEF;

[PIF,R1,EL,AZ,BSV,RELAT,TROP,dPR4,satPos_ECEF_expected,~] = singlepointsolution2(
    PRN_number,rinexDataGPS,clean_GPSbroadcast,siteguess);
n=length(PRN_number);
%Problem 2
[state_vector_corrections,post_fit_pseudorange_residual,GENU] = Gmaker(n,dPR4,siteguess,
    satPos_ECEF_expected);
detlarowS=dPR4;
fprintf('Prefit residual:\n');
disp(detlarowS);

%part c

fprintf(' state vector corrections:\n');
disp(state_vector_corrections);

%part d

fprintf('post fit pseudorange residuals:\n ');
disp(post_fit_pseudorange_residual);

%% Problem 2 e

load('NIST_RINEX_GPS.mat');
load('NIST_GPS_broadcast.mat');

PRN_number = [4;7;9;16;26];

NIST_ECEF = [-1288398.567 -4721696.932 4078625.350];
siteguess=NIST_ECEF;

[PIF,R1,EL,AZ,BSV,RELAT,TROP,dPR4,satPos_ECEF_expected,~] = singlepointsolution2(
    PRN_number,rinexDataGPS,clean_GPSbroadcast,siteguess);
n=length(PRN_number);
%Problem 2
[state_vector_corrections,post_fit_pseudorange_residual,GENU] = Gmaker(n,dPR4,siteguess,
    satPos_ECEF_expected);
detlarowS=dPR4;
```

```
fprintf('Prefit residual:\n');
disp(detlarowS);

%part c


fprintf(' state vector corrections:\n');
disp(state_vector_corrections);

%part d

fprintf('post fit pseudorange residuals:\n ');
disp(post_fit_pseudorange_residual);

function [PIF,R1,EL,AZ,BSV,RELAT,TROP,dPR4,satPos_ECEF_expected,satClkCorr] =
    singlepointsolution2(PRN_number,rinexDataGPS,clean_GPSbroadcast,siteguess)


% NIST_ECEF = [-1288398.567 -4721696.932 4078625.350];
% siteguess=NIST_ECEF;

n=length(PRN_number);

PIF=zeros(n,1);
R1=zeros(n,1);
EL=zeros(n,1);
AZ=zeros(n,1);
BSV=zeros(n,1);
RELAT=zeros(n,1);
TROP=zeros(n,1);
dPR4= zeros(n,1);
satPos_ECEF_expected=zeros(n,3);

for i=1:length(PRN_number)

PRN=PRN_number(i);
[PIFtemp,R1temp,ELtemp,AZtemp,BSVtemp,RELATtemp,TROPtemp,dPR4temp,
    satPos_ECEF_expectedtemp,satClkCorr]=singlepointsolution(PRN,rinexDataGPS,
    clean_GPSbroadcast,siteguess);

PIF(i)=PIFtemp(1);
R1(i)=R1temp(1);
EL(i)=ELtemp(1);
AZ(i)=AZtemp(1);
BSV(i)=BSVtemp(1);
RELAT(i)=RELATtemp(1);
TROP(i)=TROPtemp(1);
dPR4(i)=dPR4temp(1);
satPos_ECEF_expected(i,:)=satPos_ECEF_expectedtemp(1,:);
fprintf('PRN: %d \n',PRN)
fprintf('PIF: %.2f [m],R1: %.2f [m], Elevation: %.2f [deg], Azimuth: %.2f [deg], Clock
    error: %.2f [m], Relativity: %.2f [m], Trop: %.2f [m] \n ',PIF(i),R1(i),EL(i),AZ(i),
    BSV(i),RELAT(i),TROP(i));
```

```
end

end
function [PIF,R1,EL,AZ,BSV,RELAT,TROP,dPR4,satPos_ECEF_expected,satClkCorr]=
    singlepointsolution(PRN_number,rinexDataGPS,clean_GPSbroadcast,siteguess)


PRN_number = PRN_number;
PRN_index = rinexDataGPS.SatelliteID == PRN_number;
PRN_GPS_rinexData = rinexDataGPS(PRN_index,:);

% Reading in sp3
sp3 = read_sp3('IGS0OPSFIN_20262310000_01D_15M_ORB.SP3');
satPRN = extract_sp3_prn(sp3);

[NIST_GPS_week, NIST_GPS_TOW] = utc2gpstime(PRN_GPS_rinexData.Time);

NIST_ECEF = siteguess;

ephem_time_s = NIST_GPS_week.*604800 + NIST_GPS_TOW;
ephem_time_hr = ephem_time_s/3600;

gap_idx = [false; diff(ephem_time_s) > 1.5*median(diff(ephem_time_s))];

% Expected range corrected for light-time and Earth rotation
[R_expected, AZ_expected, EL_expected,satPos_ECEF_expected] = compute_expected_range(
    clean_GPSbroadcast, NIST_GPS_week, NIST_GPS_TOW, PRN_number, NIST_ECEF);
R_expected(gap_idx) = NaN;

% Find NIST time in hours
NIST_HR = NIST_GPS_week*604800 + NIST_GPS_TOW;

% Plotting satellite clock correction
[health,satPos,satVel,satClkCorr,~,~,relCorr] = eph2pvt2025(clean_GPSbroadcast, [
    NIST_GPS_week NIST_GPS_TOW], PRN_number);

zd = 2;
tropo_correction = tropomodel(zd, EL_expected);

f1 = 1575.42e6; % C1C frequency
f2 = 1227.6e6; % C2L frequency

% Calling ionospheric model
[PRIF, iono] = ionocorr (PRN_GPS_rinexData.C1C , f1, PRN_GPS_rinexData.C2W, f2);

PIF=PRIF;
R1=R_expected;
EL=EL_expected;
AZ=AZ_expected;
BSV=satClkCorr;
RELAT=relCorr;
TROP=tropo_correction;

dPR4 = PRIF - (R_expected - satClkCorr - relCorr + tropo_correction);
```

```
end

function [state_vector_corrections,post_fit_pseudorange_residuals,GENU] = Gmaker(n,dPR4,
    siteguess,satPos_ECEF_expected)
detlarowS=dPR4;
G=zeros(n,4);
NIST_ECEF = [-1288398.567 -4721696.932 4078625.350];
NIST_LLA=ecef2lla(NIST_ECEF);
ref_lat_deg=NIST_LLA(1);
ref_lon_deg=NIST_LLA(2);
C_ECEF2ENU = ECEF2ENU(ref_lat_deg, ref_lon_deg);

for i=1:n
rdiff=satPos_ECEF_expected(i,:)-siteguess;
rdiffENU=C_ECEF2ENU*rdiff';
ROsENU=norm(rdiffENU);
ROs=norm(rdiff);
G(i,:)=[-rdiff/ROs,1];
GENU(i,:)=[-rdiffENU'/ROsENU,1];
end

fprintf('Prefit residual:\n');
disp(detlarowS);

fprintf('Measurement sensitivity matrix:\n');
disp(G);


state_vector_corrections = (G'*G)\(G'*detlarowS);


fprintf(' state vector corrections:\n');
disp(state_vector_corrections);


post_fit_pseudorange_residuals=detlarowS-G*state_vector_corrections;


fprintf('post fit pseudorange residuals:\n ');
disp(post_fit_pseudorange_residuals);

end
```